



Universidad Autónoma de Nuevo León
Facultad de Ciencias Físico Matemáticas



PIA:

Proyecto Integrador — Sistema de Punto de Venta para Gimnasio

Tema: *Manual Técnico*

Laboratorio de Programación Orientada a Objetos

Gpo. 033
Etian Daniel Alvarado Martínez
2011642

GymJFX es una aplicación de escritorio JavaFX multihilo (multi-threaded) que cumple con el patrón MVC, diseñada para gestionar operaciones de gimnasio, facturación de suscripciones de membresías, registros de entrada (check-in) y salida (check-out), inventarios de equipos y programación de clases. El sistema implementa la serialización de Java (Java Serialization) para la persistencia, utiliza hilos de trabajo secundarios (multi-threaded workers) para la generación de reportes en segundo plano y copias de seguridad automáticas (auto-backups), e incorpora excepciones personalizadas (custom exceptions) para el manejo de errores de lógica de negocio.

1. Arquitectura del sistema

La aplicación está estructurada estrictamente de acuerdo con el patrón de arquitectura **Model-View-Controller (MVC)**:

- **Model:** Representa el estado de la aplicación y los modelos de datos de la lógica de negocio (por ejemplo, `Client`, `MembershipType`, `Equipment`, `ClassSession`, `AccessLog`). Las mutaciones de estado y los métodos de serialización están encapsulados dentro del contenedor principal `GymSystem`.
- **View:** Construida utilizando archivos XML declarativos estándar (`.fxml`), definiendo la jerarquía de los contenedores y elementos de la interfaz de usuario (botones, entradas, tablas).
- **Controller:** Gestiona eventos y entradas desde las vistas (Views), orquesta las mutaciones en los modelos (Models), activa ventanas emergentes (popups) y ejecuta tareas asíncronas.



2. Diagrama de clases UML

A continuación se presenta el diagrama de clases UML completo que mapea todos los paquetes, clases, atributos, métodos, herencia y dependencias dentro de GymJFX:

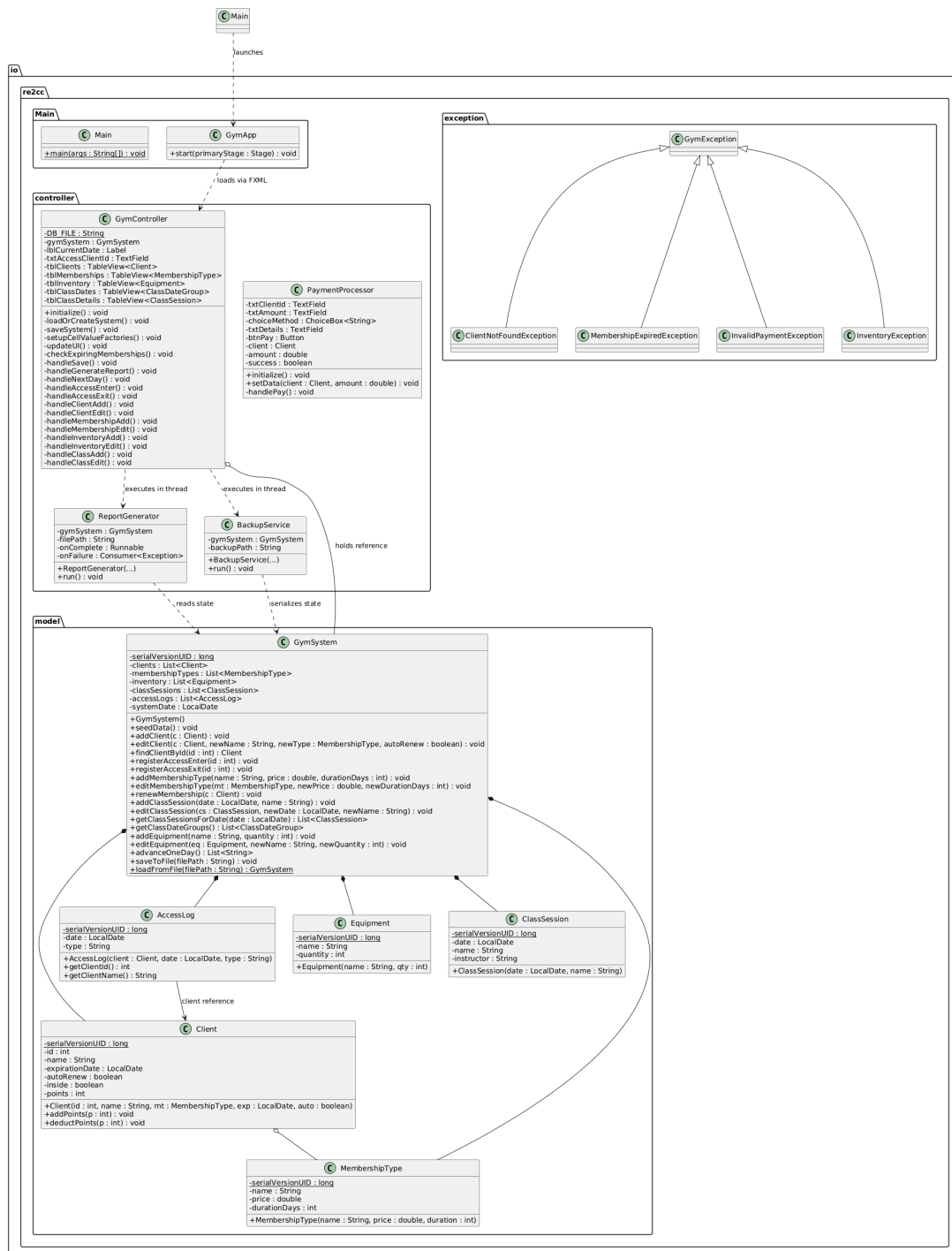


Figura 1: Diagrama de clases UML completo de la aplicación GymJFX

3. Referencia de la estructura de paquetes y clases

Paquete: `io.re2cc.Main`

- **Main.java**
 - **Propósito:** El punto de entrada estándar de la clase principal de la aplicación.
 - **Función principal:** Evita la validación del módulo JavaFX al invocar `Application.launch(GymApp.class, args)` directamente desde una clase que no extiende a `javafx.application.Application`.
- **GymApp.java**
 - **Propósito:** Clase de inicialización de la interfaz de usuario (UI) que extiende `Application`.
 - **Función principal:** Carga `MainView.fxml`, configura la escena de la ventana principal (640x400 píxeles) y muestra el escenario (stage).

Paquete: `io.re2cc.model`

- **GymSystem.java**
 - **Propósito:** Contenedor de almacenamiento centralizado que contiene colecciones de clientes, planes, equipos, clases, registros de historial de check-in y la fecha simulada.
 - **Funciones principales:**
 - `seedData()` : Inserta más de 20 registros de prueba si no hay una base de datos al iniciar la aplicación.
 - `saveToFile(path)` / `loadFromFile(path)` : Métodos que escriben/leen el flujo de datos binarios a través de `ObjectOutputStream` / `ObjectInputStream`.
 - `advanceOneDay()` : Incrementa la fecha simulada, ejecuta renovaciones automáticas de suscripción para los clientes que tienen activada la opción de auto-renovación (auto-renew), realiza el cargo a la tarjeta (apoyándose en `renewMembership`) y registra los resultados.
 - `registerAccessEnter(id)` / `registerAccessExit(id)` : Verifica los permisos de acceso, lanza excepciones personalizadas para tarjetas vencidas o registros faltantes, marca el estado de presencia interna (inside) y añade registros al historial de acceso.
- **Client.java**
 - **Propósito:** Model que representa a un miembro del gimnasio.
 - **Atributos:** ID (entero), Name (String), referencia a `MembershipType`, Expiration Date (LocalDate), bandera booleana de `autoRenew`, bandera booleana de estado de presencia interna (`inside`) y contador de puntos de fidelidad (entero).
- **MembershipType.java**
 - **Propósito:** Model que representa los planes dinámicos de membresía del gimnasio.
 - **Atributos:** Name (String), Price (double) y duración en días (entero).
- **Equipment.java**

- **Propósito:** Representa los artículos de maquinaria del gimnasio y pesas.
- **Atributos:** Nombre del artículo (String) y cantidad (entero).
- **ClassSession.java**
 - **Propósito:** Representa cursos grupales de fitness programados.
 - **Atributos:** Fecha programada (LocalDate) y nombre de la clase (String).
- **AccessLog.java**
 - **Propósito:** Registro de historial que representa los check-ins y check-outs de los clientes.
 - **Diseño:** Delega en una referencia a un objeto `Client` para obtener IDs y nombres de forma segura y dinámica, incluso si cambian.

Paquete: `io.re2cc.controller`• **GymController.java**

- **Propósito:** La clase Controller principal mapeada a `MainView.fxml`.
- **Funciones principales:**
 - `initialize()` : Desencadena la carga de la base de datos serializada `db.ser`, configura las fábricas de valores de celda utilizando propiedades lambda, escucha cambios de selección de fecha para mostrar clases e informa sobre planes próximos a vencer.
 - `handleNextDay()` : Avanza el día, activa un hilo de trabajo de copia de seguridad (backup) en segundo plano y muestra los registros.
 - `handleAccessEnter()` / `handleAccessExit()` : Valida y registra los intentos de acceso, mostrando alertas de éxito de check-in con puntos, o cuadros de error.
 - `handleClientAdd()`, `handleClientEdit()`, etc.: Instancia ventanas secundarias dinámicamente a partir de archivos FXML específicos, establece callbacks y actualiza las tablas.

• **PaymentProcessor.java**

- **Propósito:** Controller de tareas en segundo plano mapeado a `PaymentView.fxml`.
- **Funciones principales:** Gestiona validaciones de formato de tarjetas de crédito (16 dígitos numéricos), entrega de cambio en efectivo y saldos de puntos. Simula un retraso de procesamiento dentro de un hilo secundario en segundo plano.

• **ReportGenerator.java**

- **Propósito:** Clase de trabajo (worker class) que implementa `Runnable` para generar reportes financieros y resúmenes de check-in en formato de texto en un hilo en segundo plano.
- **Funciones principales:** Calcula el conteo de clientes activos, ventas totales, inventarios y escribe en `gym_report.txt` tras un tiempo de espera (sleep) simulado de 2.5 segundos.

• **BackupService.java**

- **Propósito:** Clase de trabajo (worker class) que implementa `Runnable` para guardar de forma silenciosa copias de seguridad en un archivo de serialización de respaldo.
- **Funciones principales:** Crea automáticamente una carpeta y serializa los objetos de `GymSystem` bajo `backups/` después de un retraso simulado de 1 segundo.

• **ClientAddController.java**

- **Propósito:** Controller mapeado a `ClientAddView.fxml` para registrar nuevos clientes.
- **Funciones principales:**
 - `setSystemAndCallback(system, callback)` : Recibe el estado de la base de datos y guarda un callback de actualización.
 - `handlePay()` : Valida los campos de entrada, instancia un modelo de cliente temporal, inicia la etapa de procesamiento de pagos y registra al cliente si la transacción es exitosa.

• **ClientEditController.java**

- **Propósito:** Controller mapeado a `ClientEditView.fxml` para modificar perfiles de clientes.

- ▶ **Funciones principales:**
 - `setClientSystemAndCallback(client, system, callback)` : Carga los datos del cliente seleccionado en los campos del diálogo.
 - `handleSave()` : Guarda los cambios. Si el tipo de membresía cambia, abre el diálogo de pago para el precio del nuevo plan y actualiza la fecha de vencimiento al tener éxito.
- **MembershipAddController.java**
 - ▶ **Propósito:** Controller mapeado a `MembershipAddView.fxml` para crear nuevos planes de suscripción.
 - ▶ **Funciones principales:**
 - `handleSave()` : Valida que las entradas de nombre de plan, precio y duración sean positivas y no estén vacías, y crea el plan en el sistema.
- **MembershipEditController.java**
 - ▶ **Propósito:** Controller mapeado a `MembershipEditView.fxml` para editar planes existentes.
 - ▶ **Funciones principales:**
 - `setMembershipSystemAndCallback(...)` : Deshabilita la modificación del nombre del plan y precarga el precio y la duración.
 - `handleSave()` : Valida y guarda las restricciones actualizadas del plan.
- **InventoryAddController.java**
 - ▶ **Propósito:** Controller mapeado a `InventoryAddView.fxml` para registrar nuevos equipos.
 - ▶ **Funciones principales:**
 - `handleSave()` : Valida las restricciones de entrada y registra el nuevo inventario.
- **InventoryEditController.java**
 - ▶ **Propósito:** Controller mapeado a `InventoryEditView.fxml` para modificar las cantidades de equipos existentes.
 - ▶ **Funciones principales:**
 - `handleSave()` : Restringe cantidades de inventario negativas y actualiza la cantidad de artículos.
- **ClassAddController.java**
 - ▶ **Propósito:** Controller mapeado a `ClassAddView.fxml` para programar clases de fitness.
 - ▶ **Funciones principales:**
 - `handleSave()` : Valida la fecha de la clase y el nombre del curso, programando una nueva sesión.
- **ClassEditController.java**
 - ▶ **Propósito:** Controller mapeado a `ClassEditView.fxml` para editar clases programadas.
 - ▶ **Funciones principales:**
 - `handleSave()` : Actualiza los detalles de fecha o nombre de una sesión de clase programada.

Paquete: `io.re2cc.exception`

- **GymException.java**
 - ▶ **Propósito:** Clase base para excepciones de tipo checked exception.

- **ClientNotFoundException.java**
 - **Propósito:** Se lanza cuando fallan las búsquedas de IDs de clientes no registrados.
- **MembershipExpiredException.java**
 - **Propósito:** Se lanza cuando se registra un check-in para un cliente con membresía vencida.
- **InvalidPaymentException.java**
 - **Propósito:** Se lanza si una tarjeta de crédito falla las validaciones o si el efectivo entregado es insuficiente.
- **InventoryException.java**
 - **Propósito:** Se lanza en caso de cantidades negativas o nombres de artículos de equipo duplicados.

4. Implementaciones técnicas clave de Java/FX

Multihilo y seguridad de hilos en la interfaz de usuario (UI Thread Safety)

En JavaFX, las modificaciones directas a los nodos activos desde un hilo diferente al hilo de aplicación principal de JavaFX (**JavaFX Application Thread**) desencadenan una excepción `IllegalStateException`.

GymJFX ejecuta clases de trabajo (worker classes) como `ReportGenerator` y `BackupService` en hilos de trabajo independientes (`new Thread(runnable).start()`) para evitar que la interfaz de usuario se congele (UI freezes). Las acciones de éxito o fallo que actualizan los componentes de la interfaz de usuario (como campos de texto o ventanas emergentes de alerta) se programan de manera segura de vuelta en el hilo de JavaFX anidándolas dentro de:

```
Platform.runLater(() -> {  
    // Código que modifica nodos de JavaFX o muestra alertas  
});
```

Persistencia y serialización de archivos

En lugar de escribir lógica de análisis (parsing) para archivos de configuración de texto personalizados, el estado completo de `GymSystem` se serializa en un solo archivo binario llamado `db.ser`. Todas las clases de los modelos implementan la interfaz `Serializable`. Para restaurar el estado, se leen los datos binarios, se realiza una conversión de tipo (cast) de vuelta a `GymSystem` y se cargan en el controlador principal. Si los cambios en las clases rompen la compatibilidad con versiones anteriores, la deserialización falla, recurre a un plan alternativo (fall back), captura las excepciones y genera nuevos datos semilla para garantizar la confiabilidad del programa.